

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Virtuální herbář funkcí

Marek Jirušek

Obsah

1 Úvod **3**

1.1 Prohlášení o použití umělé inteligence 3
1.2 Definice problému 3
1.3 Cíl práce 4
1.4 Struktura dokumentace 4

2 Architektura aplikace **6**

2.1 Komponenty aplikace 6
2.2 Datový tok a komunikace 6

3 Architektura dat a matematické jádro **8**

3.1 Datově řízený návrh a real-time monitoring 8
3.1.1 Úložiště a správa souborů 8
3.1.2 Mechanismus FileSystemWatcher a asynchronní aktualizace 9
3.2 Analýza matematických výrazů a kompilace do AST 9
3.2.1 Lexikální analýza (Tokenizace) 10
3.2.2 Algoritmus Shunting-yard 11
3.2.3 Kompilace do abstraktního syntaktického stromu 12
3.2.4 Detekce nespojitostí a algoritmus bisekce 14
3.2.5 Triangulace, směrové vektory a výškové ořezání 15
3.2.6 Kompletace modelu a fyzikální interakce 16

4 Optimalizace a limity enginu **17**

4.1 Indexování sítě 17
4.2 Správa paměti a eliminace alokací 18
4.3 Architektura řízená událostmi (Event-driven) 18

5 Uživatelské rozhraní, interakce a zobrazení ve VR **19**

5.1 Prostorové uživatelské rozhraní (Tabule a příruční menu) 19
5.2 Fyzikální manipulace a volný pohyb 20
5.3 Vizuální reprezentace a orientace v prostoru 21

6 Výsledky a zhodnocení **22**
6.1 Testovací prostředí a metodika 22
6.2 Snímková frekvence a stabilita (FPS) 22
6.3 Praktické ukázky vygenerovaných modelů 23
6.3.1 Spojité plochy a dynamické parametry 23
6.3.2 Zpracování singularit a dělení nulou 24

7 Závěr **26**
7.1 Shrnutí dosažených výsledků 26
7.2 Možnosti budoucího rozšíření 27

A Uživatelská příručka **28**
A.1 Hlavní tabule a výběr funkce 28
A.2 Úprava seznamu funkcí 29
A.3 Fyzická manipulace s modelem 30
A.4 Příruční menu a dynamické úpravy 31
A.5 Průzkum vnitřní topologie 32

Bibliografie **35**

Úvod

Pochopit chování funkcí o jedné proměnné je pro studenty středních škol relativně intuitivní, jelikož lze snadno vizualizovat jejich grafy na papíře nebo pomocí, v dnešní době již jednoduše dostupných, nástrojů. Chování funkcí o dvou a více proměnných však již není tak jednoduché, jelikož nejenže vyžaduje určitou představivost, ale v dnešní době neexistuje dostatečný počet dostupných nástrojů, které by umožňovali interaktivní zkoumání těchto funkcí. Tato práce vznikla zejména z důvodu digitalizovat herbář funkcí, který byl napsán na katedře matematiky. Cílem této práce bylo vytvořit virtuální herbář funkcí, který umožní studentům interaktivně prozkoumávat chování funkcí o dvou proměnných ve virtuální realitě.

1.1 Prohlášení o použití umělé inteligence

Během přípravy této práce autor použil nástroj umělé inteligence Google Gemini k následujícím účelům:

- konzultace týkající se implementačních detailů a návrhu softwarové architektury systému,
- asistence při formulaci a stylistické úpravě odborného textu a kontrola terminologické konzistence.

Po použití tohoto nástroje autor pečlivě zkontroloval a upravil veškerý obsah podle potřeby. Veškerá rozhodnutí týkající se návrhu, implementace a interpretace výsledků jsou výhradně vlastní prací autora. Autor přebírá plnou odpovědnost za výsledný obsah práce.

1.2 Definice problému

Generování 3D modelů grafů funkcí v reálném čase je náročný úkol, jelikož vyžaduje spoustu výpočtů. Navíc, pro zajištění plynulého zážitku ve virtuální realitě, je důležitý výkon a minimální odezva při překreslování grafů po změně parametrů. Aplikace musí být schopna:

- Zpracovat uživatelský vstup: Aplikace musí být schopna přijímat nové funkce, zpracovávat změny parametrů a definičního oboru a aktualizovat grafy v reálném čase.
- Generování 3D modelů: Aplikace musí být schopna generovat 3D modely grafů funkcí, které jsou vizuálně přehledné a informativní a zároveň edukativní pro studenty.
- Body nespojitosti: Aplikace by měla identifikovat a vizualizovat body nespojitosti funkcí.
- Interakce ve VR: aplikace by měla umožnit uživatelům za běhu měnit předpisy funkcí, měnit parametry a definiční obor a okamžitě vidět aktualizované grafy ve virtuální realitě bez opouštění VR prostředí.

1.3 Cíl práce

Hlavním cílem této bakalářského projektu je vyvinout aplikaci „Virtuální herbář funkcí“, která uživatelům umožní interaktivně prozkoumávat matematické funkce dvou proměnných v prostředí virtuální reality (VR). Aplikace je navržena jako názorná učební pomůcka, jež má usnadnit pochopení prostorových grafů a demonstrovat, jak se jejich tvar mění v závislosti na různých dynamických parametrech na zvoleném definičním oboru.

Pro úspěšné dosažení tohoto cíle byli stanoveny následující cíle:

- Implementace algoritmu pro převod textového zápisu funkce do podoby 3D sítě (meshe) a její plynulé generování v reálném čase.
- Návrh řešení pro detekci bodů nespojitosti a korektní ořezávání asymptot, aby nedocházelo k deformacím generovaného modelu.
- Optimalizace výpočetní náročnosti aplikace s cílem udržet stabilní snímkovou frekvenci.
- Vytvoření prostorového uživatelského rozhraní umožňující intuitivní manipulaci s modelem, úpravu parametrů rovnice a změnu definičního oboru přímo ve VR.

1.4 Struktura dokumentace

Následující text je logicky členěn do několika celků, které popisují vývoj a implementaci aplikace. Druhá kapitola představuje celkovou architekturu aplikace a způsob, jakým mezi sebou komunikují jednotlivé komponenty. Třetí kapitola poté rozebírá

matematické jádro, od zpracování textového vstupu přes kompilaci do abstraktního syntaktického stromu (AST) až po samotný algoritmus generování 3D sítě a řešení asymptot. Čtvrtá kapitola se následně věnuje optimalizačním technikám a limitům enginu, které bylo nutné překonat pro zajištění plynulého chodu ve VR. Pátá kapitola popisuje návrh prostorového uživatelského rozhraní a implementaci fyzikálních interakcí i volného pohybu pomocí rámce XR Interaction Toolkit. Poslední kapitoly se zabývají testováním a shrnutím celého projektu, včetně návrhů pro budoucí rozšíření a vylepšení.

Architektura aplikace

Architektura aplikace je strukturována do tří hlavních modulů, které spolu komunikují pomocí událostí a sdílených datových struktur. Tento přístup zajišťuje snadnou udržitelnost kódu a umožňuje plynulý chod aplikace i při výpočetně náročných operacích ve virtuální realitě.

2.1 Komponenty aplikace

Aplikace je logicky rozdělena do tří hlavních softwarových komponent, z nichž každá plní svou specifickou roli:

- **Manager:** Představuje řídicí uzel aplikace. Načítá dostupné funkce ze souborového systému, buduje interaktivní nabídku na virtuální tabuli a spravuje fyzikální interakce uživatele s modelem, jako je například obouruční škálování velikosti grafu.
- **Generator:** Tvoří matematické jádro aplikace. Přijímá textové řetězce reprezentující matematické funkce, zařizuje jejich kompilaci pomocí třídy `MathPreprocess` a následně provádí iterativní výpočet prostorových souřadnic. Využívá algoritmus bisekce pro detekci asymptot a dynamicky přepisuje geometrii v komponentě MeshFilter.
- **UIManager:** Stará se o dynamické rozhraní. Modul naslouchá událostem z `Generatoru` a na jejich základě automaticky instancuje ovládací prvky (posuvníky a číselníky) pro manipulaci s definičním oborem a nalezenými parametry rovnice.

2.2 Datový tok a komunikace

Aby nedocházelo k propadům snímkové frekvence, není komunikace mezi moduly vázana na pravidelnou aktualizační smyčku enginu, ale je striktně řízena událostmi (event-driven architecture).

Zpracování nové funkce probíhá asynchronně. Jakmile uživatel vybere funkci, `Manager` předá její předpis komponentě `Generator`, která provede lexikální analýzu, kompilaci a vygenerování počáteční 3D sítě. Následně `Generator` vyvolá událost `OnFunctionLoaded`, kterou zachytí `UIManager` a dynamicky podle ní sestaví příslušné ovládací prvky pro zachycené parametry.

Pokud následně uživatel pomocí paprsku ovladače změní hodnotu parametru, `UIManager` zapíše novou hodnotu přímo do pole parametrů v `Generatoru` a explicitně zavolá metodu `RefreshGraph()`. Aplikace tak negeneruje síť neustále, ale výhradně v okamžiku, kdy dojde ke skutečné změně vstupních dat.

Architektura dat a matematické jádro

Tato kapitola se věnuje vnitřní logice aplikace, která zajišťuje převod textových řetězců s matematickými výrazy do podoby interaktivních 3D modelů ve virtuální realitě. Architektura je datově řízená (data-driven), tím pádem veškeré předpisy funkcí jsou uloženy externě a aplikace je dynamicky zpracovává až za běhu aplikace. Toto řešení umožňuje uživatelům průběžně modifikovat obsah virtuálního herbáře bez nutnosti zásahu do zdrojového kódu či restartování samotné aplikace, což usnadňuje proces výuky a experimentování.

3.1 Datově řízený návrh a real-time monitoring

Základním architektonickým principem řešení je striktní oddělení výpočetní logiky od spravovaných dat. Místo statického definování tzv. *hard-coded* funkcí přímo v modulu `Generator` využívá aplikace externí textový konfigurační soubor s názvem `functions.txt`, ve kterém je definován seznam rovnic tvořících obsah herbáře.

3.1.1 Úložiště a správa souborů

Centrálním úložištěm pro veškeré matematické definice je soubor `functions.txt`, který je alokován ve speciálním adresáři `StreamingAssets`. Tato složka je z pohledu enginu Unity specifická tím, že její obsah nepodléhá interní kompilaci a zůstává v nezměněné podobě i po sestavení výsledné aplikace (buildu). Koncoví uživatelé, typicky vyučující, tak mohou snadno doplňovat či upravovat vyučovanou látku zcela nezávisle na vývojovém cyklu programu.

Při startu aplikace provádí řídicí třída `Manager` kontrolu integrity datového úložiště pomocí metody `EnsureFileExistance()`. Pokud aplikace detekuje, že konfigurační soubor chybí nebo je poškozen, automaticky vygeneruje nový a zainicializuje jej sadou předdefinovaných demonstračních rovnic. Pro maximální zjednodušení uživatelského přístupu navíc aplikace při prvním spuštění vytvoří v kořenovém adresáři programu zástupce ve formě dávkového souboru `1_Upravit_Rovnice.bat`,

který zajistí okamžité otevření konfiguračního souboru ve výchozím textovém editoru hostitelského operačního systému.

3.1.2 Mechanismus FileSystemWatcher a asynchronní aktualizace

Aby bylo možné modifikovat rovnice za běhu aplikace, využívá třída `Manager` standardní .NET komponentu `FileSystemWatcher`. Tento nástroj na pozadí monitoruje cílový adresář a naslouchá události zápisu (`NotifyFilters.LastWrite`) do souboru `functions.txt`.

Jelikož události ze sledování souborového systému probíhají ve vedlejším výpočetním vlákně a aplikační rozhraní enginu Unity není takzvaně *thread-safe* (bezpečné pro přímý přístup z více vláken), je proces aktualizace rozdělen do dvou synchronizovaných fází:

- **Asynchronní detekce:** Jakmile dojde k uložení změn v textovém souboru, `FileSystemWatcher` zachytí událost `OnFileChanged`, v níž pouze nastaví logický příznak `needsRefresh` na hodnotu `true`.
- **Synchronizace v hlavním vlákně:** Vestavěná metoda `Update()`, která se spouští v rámci hlavní renderovací smyčky enginu, tento příznak pravidelně kontroluje. Je-li detekována jeho změna, dojde k zavolání metody `LoadFunctions()`. Ta načte aktuální data ze souboru a následně vyvolá překreslení (`RefreshBoardUI()`), čímž se vyčistí stávající nabídka a na virtuální tabuli se vygenerují nové interaktivní přepínače pro každou nalezenou funkci.

Díky tomuto neblokujícímu přístupu může vyučující na připojeném počítači upravovat seznam probíraných funkcí v reálném čase, zatímco student ve virtuální realitě okamžitě vidí aktualizovanou nabídku, aniž by musel aplikaci opouštět či sundávat brýle.

3.2 Analýza matematických výrazů a kompilace do AST

Jakmile je textová reprezentace funkce (např. výraz $a \cdot \sin(b \cdot x)$) načtena, je předána třídě `MathPreprocess`. Ta zajišťuje její převod do reverzní polské notace (RPN) a následnou kompilaci do abstraktního syntaktického stromu (AST).

Vzhledem k tomu, že vstupem je textový řetězec, nasazení prostého interpretu by bylo pro potřeby aplikace zcela nedostačující. Opakované vyhodnocování textu pro desítky tisíc bodů sítě v každém vykreslovaném snímku by vedlo k enormní výpočetní režii a drastickému propadu snímkové frekvence, což je ve virtuální realitě

kritický problém způsobující nevolnost. Zpracování výrazu je proto rozděleno do následujících tří fází:

- **Lexikální analýza (Tokenizace):** Rozdělení vstupního řetězce na elementární textové a sémantické jednotky (tzv. tokeny).
- **Shunting-yard algoritmus:** Syntaktická analýza tokenů a jejich převod z infixové notace do RPN, čímž je jednoznačně určena priorita operací bez nutnosti použití závorek.
- **Kompilace do AST:** Konstrukce syntaktického stromu z RPN a jeho překlad do nativního kódu (do podoby delegátu) pro dosažení maximální rychlosti při samotném výpočtu bodů.

3.2.1 Lexikální analýza (Tokenizace)

Než může být matematický výraz algoritmicky zpracován, musí být rozložen na základní stavební bloky, tzv. tokeny. Úkolem metody `Tokenize` je lineárně projít řetězec a rozdělit jej na jednotky, se kterými lze následně pracovat.

Před samotným průchodem je text očištěn od bílých znaků (mezer), převeden na malá písmena a případné desetinné čárky jsou normalizovány na tečky, čímž se zajistí kompatibilita s parsováním datového typu `float` v jazyce C#. Následně algoritmus iteruje řetězcem znak po znaku a shlukuje je do následujících tří kategorií:

- **Numerické hodnoty:** Narazí-li analyzátor na číslici nebo desetinnou tečku, shlukuje všechny bezprostředně následující platné znaky do jediného číselného tokenu.
- **Alfanumerické řetězce (Slova):** Detekuje-li analyzátor písmeno, spojí souvislou řadu písmen do jednoho textového tokenu. V případě výrazu $a \cdot \sin(b \cdot x)$ tak budou rozpoznány samostatné textové jednotky `a`, `sin`, `b` a `x`.
- **Operátory a závorky:** Jednotlivé znaky z předem definované množiny `+-*/^()` jsou izolovány jako samostatné matematické či strukturní operátory.

Kromě základního rozdělování textu se analyzátor v této fázi elegantně vypořádává s problematickým unárním mínusem, který se v matematických výrazech běžně vyskytuje (např. ve výrazech `-x`, `-(x+y)`, nebo `sin(-x)`). Analyzátor tuto situaci uvnitř cyklu řeší zavedením jednoduchého pravidla: pokud je detekován znak `-` na samém začátku výrazu nebo bezprostředně za otevírací závorkou `()`, je automaticky před tento znak vložen nový token `0`. Tím dochází k internímu převodu unární operace na operaci binární (ve smyslu $0 - x$), což efektivně a bezpečně předchází pádům při vyhodnocování priorit v navazujícím algoritmu Shunting-yard.

3.2.2 Algoritmus Shunting-yard

Výstupem předchozí fáze je seznam tokenů v běžné infixové notaci. Tato notace je přirozená pro člověka, pro přímé strojové vyhodnocení je však nevhodná z důvodu nutnosti explicitně řešit priority operátorů a uzávorkování. Pro převod výrazu do reverzní polské notace (RPN) implementuje třída `MathPreprocess` modifikovaný algoritmus ***Shunting-yard*** [Dij61].

V RPN se operátory zapisují za své operandy, čímž odpadá nutnost použití závorek. Metoda `ShuntingYard` pro tento převod využívá dvě datové struktury: výstupní seznam (`List<string> outputQueue"`) a dočasný zásobník (`operatorStack`). Prioritu operátorů definuje pevný slovník; sčítání a odčítání mají nejnižší prioritu (1), umocňování nejvyšší (3).

Algoritmus zpracovává tokeny zleva doprava a operandy (čísla, proměnné) vkládá přímo do výstupu. Operátory a matematické funkce odkládá na dočasný zásobník, dokud není jednoznačně určena jejich priorita vůči dalším operacím. Při načtení matematického operátoru dojde k jeho porovnání s operátorem na vrcholu zásobníku. Pokud má operátor na zásobníku vyšší nebo stejnou prioritu, přesune se do výstupního seznamu. Výjimkou je operátor mocniny (^), který je asociativní zprava doleva.

Závorky plní roli bariér. Otevírací závorka se ukládá na zásobník. Při nalezení zavírací závorky se všechny operátory na zásobníku až po první otevírací závorku přesunou na výstup. Po zpracování všech vstupních tokenů se zbylé operátory ze zásobníku vyprázdní do výstupu. Tím vzniká kompletní RPN reprezentace zadané funkce, připravená pro kompilaci do abstraktního syntaktického stromu (AST).

Oproti standardní verzi algoritmu, která zpracovává primárně základní aritmetické operátory a číselné konstanty, obsahuje použitá implementace dvě klíčové modifikace. První úpravou je podpora proměnných (například souřadnic x a z). Ty lexikální analyzátor identifikuje a algoritmus je následně propouští přímo na výstup jako běžné operandy. Druhým rozšířením je nativní zpracování matematických funkcí (např. goniometrických nebo logaritmických). Tyto funkce se dočasně ukládají na zásobník a k jejich přesunu do výstupní fronty dochází cíleně po zpracování zavírací závorky argumentu. Tím je algoritmicky zaručeno správné zřetězení funkce s jejím vnitřním výrazem, což standardní Dijkstrův návrh nativně neřeší.

Zdrojový kód 3.1: Strukturální pseudokód algoritmu Shunting-yard

```
foreach (string token in vstupniTokeny) {
    if (JeOperand(token)) {
        outputQueue.Add(token);
    } else if (JeFunkce(token)) {
        operatorStack.Push(token);
    } else if (JeOperator(token)) {
        while (operatorStack.Count > 0 && operatorStack.Peek() != "(") {
```

```
            string topOp = operatorStack.Peek();
            if (token != "^" && Priorita(token) <= Priorita(topOp)) {
                outputQueue.Add(operatorStack.Pop());
            } else {
                break;
            }
        }
        operatorStack.Push(token);
    } else if (token == "(") {
        operatorStack.Push(token);
    } else if (token == ")") {
        while (operatorStack.Count > 0 && operatorStack.Peek() != "(") {
            outputQueue.Add(operatorStack.Pop());
        }
        if (operatorStack.Count > 0 && operatorStack.Peek() == "(") {
            operatorStack.Pop(); // Zahození levé závorky
        }
        if (operatorStack.Count > 0 && JeFunkce(operatorStack.Peek())) {
            outputQueue.Add(operatorStack.Pop());
        }
    }
}

while (operatorStack.Count > 0) {
    outputQueue.Add(operatorStack.Pop());
}
```

3.2.3 Kompilace do abstraktního syntaktického stromu

Závěrečnou fází předzpracování funkce je transformace fronty v reverzní polské notaci do podoby optimalizovaného spustitelného delegátu. Cílem této fáze je sestavit v paměti takovou strukturu, kterou bude procesor schopen opakovaně a maximálně efektivně vykonávat v reálném čase.

3.2.3.1 Abstraktní syntaktický strom (AST)

Abstraktní syntaktický strom (z anglického ***Abstract Syntax Tree***, zkráceně AST) je hierarchická stromová struktura reprezentující logickou stavbu zdrojového kódu nebo matematického výrazu. Vnitřní uzly stromu tvoří operátory (například sčítání či násobení) a matematické funkce (například sinus nebo absolutní hodnota). Listy stromu představují operandy, tedy číselné konstanty nebo proměnné.

Výhodou AST je absence nadbytečných syntaktických prvků, jako jsou závorky, jelikož posloupnost vyhodnocování je dána samotnou hierarchií stromu. Výpočet probíhá rekurzivně od nejnižších listů směrem ke kořeni.

3.2.3.2 Konstrukce stromu a kompilace v aplikaci

Proces stavby AST probíhá lineárním průchodem RPN fronty za využití knihovny System.Linq.Expressions [Mic24] a pomocného zásobníku typu `Stack<Expression>`. Algoritmus postupně odebírá tokeny a podle jejich typu vytváří příslušné uzly:

- **Konstanty a proměnné:** Číselné hodnoty jsou transformovány na uzly typu `ConstantExpression`. Prostorové proměnné x a z jsou definovány jako uzly `ParameterExpression`, sloužící jako argumenty pro iterativní výpočet.
- **Parametry (koeficienty):** Pokud analyzátor narazí na identifikátor, který nepatří mezi interní funkce (například koeficienty a, b), přiřadí mu index v dynamickém poli parametrů. Přístup k těmto hodnotám je reprezentován uzlem `Expression.ArrayIndex` nad polem `argsParam`.
- **Aritmetické operátory:** Při detekci operátoru dojde k vyjmutí dvou předchozích výrazů ze zásobníku a jejich spojení do nového uzlu (například `Expression.Add`). Vzniklý podstrom je uložen zpět na zásobník.
- **Mapování funkcí a reflexe:** Vzhledem k tomu, že použitá knihovna neobsahuje nativní uzly pro goniometrické funkce nebo mocniny, využívá kompilátor mechanismus reflexe jazyka C#. Pomocí metody `GetMethod` jsou vyhledány odpovídající definice z knihovny UnityEngine.Mathf (například `Sin`, `Pow`). Tyto operace jsou do stromu vloženy jako uzly typu `MethodCallExpression`.

Výsledkem procesu je kořenový uzel stromu zapouzdřený v objektu `LambdaExpression`. Voláním metody `.Compile()` dochází k JIT (Just-In-Time) kompilaci do mezijazyka ***Intermediate Language*** (IL). Tím vzniká spustitelný předpis typu `Func<float, float, float[], float>`. Metoda `Compile` vrací tento předpis společně se seznamem názvů nalezených koeficientů, který aplikace využívá pro dynamické generování ovládacích prvků v uživatelském rozhraní.

3.2.3.3 Optimalizace zobrazení pomocí pomocné paměti (Caching)

Nejprostším způsobem vyhodnocení matematického výrazu by bylo naprogramování ***interpretu***, který by pro každý počítaný bod sítě znovu analyzoval a zpracovával vstupní textový řetězec. Vzhledem k tomu, že 3D síť grafu může běžně obsahovat i více než 100 000 vrcholů a prostředí virtuální reality vyžaduje plynulé překreslování s frekvencí 72 až 90 Hz, musel by takový interpret provádět několik zbytečných textových operací každou vteřinu. To by vedlo k extrémní výpočetní režii a naprosté degradaci plynulosti obrazu.

Dynamická kompilace do IL kódu tuto zátěž prakticky zcela eliminuje. Zkompilovaný delegát je procesorem vykonáván nativně a s výkonem plně srovnatelným s kódem, který by programátor napevno zapsal do zdrojového souboru.

Aby se zamezilo kompilování již využívaných předpisů, využívá třída `Generator` vnitřní mezipaměť (***Caching***). Každý úspěšně zkompilovaný předpis je společně se seznamem nalezených koeficientů trvale uložen do hashovací tabulky `Dictionary`, konkrétně do privátní kolekce `compiledCache`. Pokud se uživatel k dané rovnici v herbáři později vrátí, aplikace ji okamžitě načte přímo z paměti v její přeložené podobě. Tento přístup garantuje rychlou odezvu uživatelského rozhraní i při rychlém přepínání mezi složitými matematickými modely na virtuální tabuli.

3.2.4 Detekce nespojitostí a algoritmus bisekce

Většina matematických funkcí má složité chování, jako jsou například body nespojitosti, mezery v definičním oboru nebo asymptoty (např. $f(x, y) = \frac{x^2 \cdot y^2}{x^3 + y^2}$). V místech takových nespojitostí vznikají při naivním generování sítě vizuální artefakty v podobě nepřirozeně natažených a strmých polygonů. Pro korektní zobrazení těchto jevů využívá aplikace vícefázový vyhodnocovací proces doplněný o numerickou metodu ***bisekce*** (půlení intervalu).

Generování sítě proto neprobíhá naslepo. Každý vypočítaný bod prochází validací. Pokud zkompilovaná funkce vrátí neplatnou hodnotu (`NaN` nebo nekonečno) nebo vypočítaná výška přesáhne povolený limit prostoru (`limitY`), označí algoritmus daný vrchol jako neplatný.

Následně algoritmus prozkoumá čtyři bezprostřední sousedy tohoto bodu v mřížce. Pokud alespoň jeden ze sousedů leží v platné oblasti, znamená to, že se výpočet nachází na okraji asymptoty či trhliny a spouští se proces bisekce. Princip hledání přesné hrany spočívá v postupném půlení vzdálenosti mezi platným a neplatným vrcholem:

1. Algoritmus určí středový bod (`mid`) úsečky spojující platný a neplatný bod.
2. Tento středový bod je obratem vyhodnocen pomocí zkompilovaného delegátu.
3. Pokud má středový bod platnou výšku a shodné znaménko s původním platným bodem, posouvá se platná hranice přímo do tohoto středu.
4. V opačném případě (bod je stále neplatný nebo jeho výpočet odskočil do opačného nekonečna) se do středu posouvá neplatná hranice.

Tento cyklus se standardně zopakuje desetkrát. Původně neplatný bod je následně ukotven přesně na tyto nově spočítané souřadnice.

3.2.5 Triangulace, směrové vektory a výškové ořezání

Ukotvení bodů na hranu asymptoty řeší pouze přesnou topologii vrcholů. Neřeší však samotný problém nechtěného propojování polygonů napříč touto trhlinou. Z toho důvodu si algoritmus během bisekce u každého označeného bodu ukládá směrové příznaky `dirX` a `dirZ`. Tyto vektory indikují, ze kterého směru byl původně neplatný bod přitažen ke svému platnému sousedovi. Pokud byl například bod přitažen vlevo (`dirX = -1`), znamená to, že fyzicky patří k levému břehu asymptoty.

Ve fázi triangulace, kdy se vrcholy spojují do finálních trojúhelníků, generátor tyto směrové vektory kontroluje. Základní plošný segment sítě je tvořen čtveřicí vrcholů (čtvercem). Algoritmus zkoumá, zda některý z vrcholů nebyl přitažen ze směru, který míří pryč od středu aktuálně řešeného čtverce. Pokud ano, znamená to, že se daný čtverec snaží překlenout mezeru v grafu. Algoritmus v takovém případě jeho vykreslení přeskočí příkazem `continue`, čímž tyto defektní polygony vyřadí.

Navíc se nezávisle ověřuje, zda maximální výškový rozdíl mezi rohy čtverce nepřesahuje kritickou toleranci a zda funkce v samotném středu čtverce nevrací neplatnou hodnotu. Zjednodušená logika této topologické ochrany je znázorněna v následujícím pseudokódu:

Zdrojový kód 3.2: Logika zamezení propojování polygonů přes asymptoty

```
// Prochazeni mrizky a ziskani rohu aktualniho ctverce:
// DL (Dolni—Levy), DP (Dolni—Pravy), HL (Horni—Levy), HP (Horni—Pravy)

// 1. Zahozeni ctvercu, ktere obsahuji zcela neplatne body (hodnota 999)
if (DL.y == 999 || DP.y == 999 || HL.y == 999 || HP.y == 999) continue;

// 2. Kontrola smerovych vektoru z bisekce (—1 pro vlevo/dolu, 1 pro
    vpravo/nahoru)
// Pokud byl vrchol pritazen z opacne strany, nez lezi stred ctverce,
    propast je uvnitr.
if (DL.dirX == —1 || DL.dirZ == —1) continue;
if (DP.dirX ==  1 || DP.dirZ == —1) continue;
if (HL.dirX == —1 || HL.dirZ ==  1) continue;
if (HP.dirX ==  1 || HP.dirZ ==  1) continue;

// 3. Kontrola extremni strmosti
if ((MaxVyska(DL, DP, HL, HP) — MinVyska(DL, DP, HL, HP)) > maxDifference
    ) continue;

// 4. Pojistna kontrola stredu ctverce (odhaleni velmi uzkych asymptot)
float stredY = VypoctiFunkciV(stredX, stredZ);
if (float.IsNaN(stredY) || stredY > limitY) continue;

VytvorTrojuhelnikyCtverce(DL, DP, HL, HP);
```

Ačkoliv je výše zmíněnými kroky zajištěno, že nevznikají polygony překračující trhliny, je nutné korektně zpracovat i samotné vrcholy směřující k nekonečnu. Tyto okrajové platné vrcholy jsou fyzicky oříznuty na maximální povolenou výšku (`limitY`) pomocí metody `Mathf.Clamp`. Aby na těchto oříznutých místech nevznikaly nepřirozené vizuální stropy, je výsledná síť doplněna o speciální grafický materiál (shader). Ten v kombinaci s fyzickým ořezáním zajišťuje, že se trojúhelníky blížící se k okraji limitu vykreslují postupně zprůhledněné nebo jsou zcela zahozeny. Vytváří se tak plynulý a přirozený dojem funkce mizející v nekonečnu.

3.2.6 Kompletace modelu a fyzikální interakce

Vrcholy, které se nacházejí hluboko v neplatné oblasti a nebylo možné je zachránit algoritmem bisekce, si dočasně ponechávají přiřazenou extrémní výškovou hodnotu (například `999f`). Těsně před finálním odesláním struktury do paměti grafické karty jsou tyto neviditelné vrcholy přesunuty přesně do lokálního počátku souřadného systému (`Vector3.zero`). Jedná se o vrcholy, které nakonec nejsou součástí žádného vykreslovaného trojúhelníku.

Tato normalizace datových polí je kritická z hlediska enginu pro výpočet fyziky. Jakmile je trojrozměrná síť kompletně vygenerována, je nad ní automaticky sestavena kolizní obálka `BoxCollider`. Ta je nezbytná pro správné vyhodnocování interakcí ve virtuální realitě. Její prostorové rozměry (`mesh.bounds.size`) a střed (`mesh.bounds.center`) jsou dynamicky vypočteny na základě reálných prostorových extrémů viditelného grafu.

Schování neplatných bodů do lokálního počátku zajišťuje, že se vypočtená obálka nebude uměle natahovat do nekonečna. Díky tomuto kroku odpovídá fyzikální těleso vizuální reprezentaci grafu. Uživatel tak může ve virtuální realitě výsledný matematický objekt fyzicky uchopit a plynule s ním manipulovat prostřednictvím vývojového rámce XR Interaction Toolkit.

Optimalizace a limity enginu

Vývoj aplikací pro virtuální realitu je specifický tím, že vyžaduje nejen vysoký výpočetní výkon, ale i efektivní správu paměti a vysokou optimalizaci. Zatímco u běžných desktopových aplikací je občasný pokles snímkové frekvence pouze estetickým defektem, ve virtuální realitě vede jakékoliv zpoždění při vykreslování (tzv. ***frame drop***) k závažným problémům. Může způsobovat nevolnosti a výrazně tak zhoršit celkový uživatelský zážitek.

Jelikož aplikace generuje v reálném čase modely o velikosti přesahující 10 000 vrcholů a tyto modely uživatel dynamicky mění, bylo nutné přistoupit k optimalizaci na několika úrovních. Optimalizace se dotkla samotné architektury kódu, použitých datových struktur i specifických funkcí enginu Unity.

4.1 Indexování sítě

Základem každého 3D objektu v enginu Unity je datová struktura `Mesh`. Ta se skládá z pole vrcholů (`vertices`) a pole indexů (`triangles`), jež tyto vrcholy spojují do trojúhelníkových ploch.

Z výkonnostních důvodů využívá engine Unity ve výchozím nastavení 16bitový formát pro indexování vrcholů. Tento formát pojme maximálně 2^{16} unikátních hodnot, což striktně omezuje velikost jednoho trojrozměrného modelu na 65 536 vrcholů. Pro vizualizaci některých matematických funkcí, zejména těch s vysokou frekvencí změn nebo s komplexními tvary, je však potřeba generovat sítě s mnohem vyšším rozlišením.

Aby bylo možné tento hardwarový limit překonat, aplikace modifikuje vnitřní nastavení geometrické sítě. Před samotným naplněním daty je formát indexů přepnut do 32bitového režimu:

Zdrojový kód 4.1: Aktivace 32bitového indexování geometrie

```
1 mesh.indexFormat = UnityEngine.Rendering.IndexFormat.UInt32;
```

Toto nastavení posouvá teoretický limit jednoho objektu na více než 4 miliardy vrcholů. Díky tomuto kroku není rozlišení vizualizované mřížky limitováno samot-

ným enginem, ale pouze reálným výpočetním výkonem hardwaru. To umožňuje škálovatelnost kvality zobrazení podle možností dané grafické karty a aktuálních požadavků uživatele.

4.2 Správa paměti a eliminace alokací

Dalším kritickým místem (***bottleneck***) při dynamickém generování 3D modelů je správa operační paměti a činnost tzv. ***Garbage Collectoru*** (GC). Pokud by aplikace při každé změně parametru (např. při posunutí posuvníku o jeden milimetr) vytvořila v paměti zcela nový objekt `Mesh` a nová pole pro vrcholy a indexy, došlo by nejen k rychlému vyčerpání dostupné paměti, ale i k častým zásahům GC, které by způsobovaly závažné poklesy snímkové frekvence a zhoršovaly by plynulost zobrazení.

Aplikace proto recykluje již existující objekty. Objekt `Mesh` je v paměti alokován pouze jednou, a to při inicializaci. Při požadavku na překreslení grafu se nevytváří objekt nový, ale existující struktura je pouze vyčištěna voláním metody `mesh.Clear()` a následně jsou do ní nalita nová, přepsaná data.

Obdobně je optimalizován i výpočet normálových vektorů. Namísto manuálního počítání kolmic pro každý trojúhelník aplikace využívá vestavěnou funkci `mesh.RecalculateNormals()`, která je optimalizována pro rychlé zpracování velkých množství dat a minimalizuje režii spojenou s výpočtem normál.

4.3 Architektura řízená událostmi (Event-driven)

Jak již bylo naznačeno v předchozích kapitolách, další vrstvou optimalizace je odstranění výpočtů z hlavní renderovací smyčky (metody `Update`). Celá architektura je proto řízena událostmi.

Zatímco vizuální systém ve VR běží neustále a překresluje obraz, matematické jádro spí. Až ve chvíli, kdy uživatel změní stav UI prvku, zachytí `Manager` tuto událost, aktualizuje potřebné proměnné a zavolá metodu generátoru. Tento přístup garantuje, že procesorový čas je spotřebováván výhradně v momentech, kdy dochází k reálné změně dat, a po zbytek času je plný výkon k dispozici pro potřeby VR enginu.

Uživatelské rozhraní, interakce a zobrazení ve VR

Návrh uživatelského rozhraní ve virtuální realitě vyžaduje oproti tradičním desktopovým aplikacím zcela odlišný přístup. Uživatel nevyužívá standardní vstupní zařízení, jako jsou myš a klávesnice, ale spoléhá se výhradně na prostorovou manipulaci. Pro implementaci těchto interakčních mechanismů aplikace využívá balíček XR Interaction Toolkit (XRIT) [Uni25] , který je oficiální součástí herního enginu Unity.

Tento balíček poskytuje abstrakci pro mapování vstupů z různých hardwarových ovladačů (například Meta Quest, HTC Vive nebo Valve Index) a zajišťuje jejich následný převod na interakční události.

5.1 Prostorové uživatelské rozhraní (Tabule a příruční menu)

Uživatelské rozhraní aplikace je ukotveno přímo do prostoru 3D scény ve formě interaktivních panelů (tzv. ***World Space Canvas***). Z hlediska logiky a přístupnosti se dělí na dva hlavní celky:

- **Hlavní tabule (Výběr funkce):** Tento statický panel je umístěný ve virtuálním prostoru a slouží k výběru analyzované rovnice. Třída `Manager` načítá předpisy funkcí z externího textového souboru a dynamicky je vykresluje jako seznam interaktivních tlačítek na ploše tabule. Uživatel namíří na tlačítko a stisknutím volí konkrétní funkci, kterou chce vizualizovat.
- **Příruční menu (Parametry a definiční obor):** Pro manipulaci s vygenerovaným grafem má uživatel k dispozici interaktivní menu. To je vyvolávané stiskem příslušného tlačítka na virtuálním levém ovladači. Tento prvek spravuje třída `UIManager` a dělí se na dvě sekce:

- *Menu pro úpravu parametrů:* Obsahuje dynamicky generované posuvníky (***Slidery***), které odpovídají parametrům detekovaným v aktuálně zvolené rovnici (například a, b, c).
- *Menu definičního oboru:* Obsahuje krokovací tlačítka (***Steppery***) pro přesnou úpravu definičního oboru na osách x a z ($X_{min}, X_{max}, Z_{min}, Z_{max}$). K dispozici je navíc přepínač *Uniform*, který umožňuje symetrické roztahování definičního oboru do všech směrů současně.

Pro interakci s těmito panely se používá komponenta XR Ray Interactor, která je připojena k virtuálním reprezentacím obou ovladačů. Tato komponenta vrhá do scény virtuální laserový paprsek (***Raycast***). Jakmile paprsek protne prvek uživatelského rozhraní, aplikace zaregistruje kolizi a umožní s daným prvkem manipulovat pomocí tlačítek na ovladači. Díky architektuře řízené událostmi je jakýkoliv vstup z uživatelského rozhraní (například pohyb posuvníku v příručním menu) okamžitě detekován a následně reflektován plynulým překreslením 3D grafu voláním metody `RefreshGraph()`.

5.2 Fyzikální manipulace a volný pohyb

Aby bylo možné zkoumat topologii vygenerovaných funkcí ze všech úhlů, podporuje aplikace fyzický úchop grafu a volný pohyb uživatele v prostoru. Kolem každého vygenerovaného grafu je automaticky obalena komponenta `BoxCollider`. Její střed a rozměry jsou po každé kompilaci dynamicky přepočítávány na základě aktuálních hranic (***bounds***) vygenerované sítě. Na tento kolizní objekt je navázaná komponenta `XRGrabInteractable`, která tvoří přímé rozhraní mezi uživatelovou rukou a 3D sítí grafu.

Logika interakce funguje na následujících principech:

- **Přímý úchop (Direct Interaction):** Pokud uživatel fyzicky přiblíží svůj virtuální ovladač k vygenerovanému grafu a stiskne úchopové tlačítko (***Grip Button***), objekt se stane dočasným potomkem jeho ruky. Uživatel tak může graf zvednout nebo s ním libovolně rotovat.
- **Úchop na dálku (Ray Interaction):** Graf je možné uchopit a přesouvat i na dálku pomocí virtuálního laserového paprsku.
- **Manuální změna měřítka (Scaling):** Pokud uživatel stiskne obě primární tlačítka současně, aktivuje se ve třídě `Manager` režim obouručního škálování. V tomto stavu může uživatel pomocí pohybu obou rukou od sebe nebo k sobě graf plynule zvětšovat či zmenšovat.

- **Zohlednění transformací a reset pozice:** Jakmile uživatel objekt pustí, rámec zruší vazbu na ovladač. Graf následně zůstane v prostoru ukotven přesně v pozici a rotaci, ve které byl zanechán. Aplikace zároveň zajišťuje bezpečné resetování pozice v případě, že by objekt propadl mimo vymezenou herní plochu (například skrz podlahu).
- **Prostupné prostředí:** Uživatel není vázán pouze na statické pozorování modelu zvenčí.Aplikace umožňuje uživateli volně procházet i skrz samotný graf.

Díky tomu, že algoritmus bisekce přesouvá veškeré vadné vrcholy mířící do nekonečna bezpečně do lokálního počátku (`Vector3.zero`), odpovídá velikost dynamicky upraveného kolizního tělesa `BoxCollider` reálné vizuální podobě grafu. Nedochází tak k matoucím a nechtěným kolizím v prázdném prostoru.

5.3 Vizuální reprezentace a orientace v prostoru

Pro správné pochopení matematického modelu je nezbytné, aby uživatel neztratil orientaci ani při libovolném natočení grafu. Vizuální prezentace modelu je proto podpořena následujícími prvky:

- **Souřadnicové osy:** Každý vygenerovaný graf je obalen vizualizací základních prostorových os (x, y, z). Tyto osy, včetně číselných popisků, jsou dynamicky generovány metodou `DrawKSS()`. V hierarchii scény jsou ukotveny jako přímí potomci samotného grafu a fyzicky vnořeny pod prázdný objekt `KSS\_Root`. K textové reprezentaci hodnot na osách je využívána komponenta TextMeshPro.
- **Oboustranné vykreslování:** Pro vykreslování trojúhelníkové sítě byl vytvořen vlastní materiál, ***shader***. Ten primárně zajišťuje oboustranné vykreslování polygonů (***Double-sided rendering***). Tato vlastnost je zcela nezbytná pro zachování plné viditelnosti grafu i při zkoumání a procházení modelu z jeho vnitřní strany.
- **Dithering a optimalizovaná průhlednost:** Aby okraje grafu a oříznuté asymptoty nevypadaly jako hrubě useknuté stěny, využívá materiál techniku ***ditheringu*** (šumového rozptylu pixelů). Při běžném poloprůhledném prolínání (***alpha blending***) dochází k vizuálnímu rozmazání a ztrátě detailů, zejména v místech, kde se nachází tenké struktury nebo ostré hrany. To by vedlo ke ztrátě hloubkové orientace. Dithering tento problém eliminuje tím, že některé pixely zcela zahazuje, čímž udržuje orientaci prostorové hloubky.

Výsledky a zhodnocení

Tato kapitola se věnuje reálnému běhu vyvinuté aplikace, hodnocení jejího výkonu a celkové analýze zvolených programátorských přístupů. Hlavním kritériem úspěchu byla schopnost aplikace plynule vykreslovat a upravovat rozsáhlé matematické modely v prostředí virtuální reality.

6.1 Testovací prostředí a metodika

Pro objektivní zhodnocení výkonu probíhalo testování na hardwarové sestavě odpovídající střední třídě herních počítačů. Tato konfigurace poskytuje dostatečný výkon pro plynulý běh virtuální reality a zároveň umožňuje odhalit případné úzké hrdla neoptimalizovaného kódu.

Hardwarová specifikace testovací stanice:

- **Grafická karta (GPU):** NVIDIA GeForce RTX 3060 Ti
- **Procesor (CPU):** AMD Ryzen 7 5700G
- **Operační paměť (RAM):** 16 GB

Zobrazení a interakce probíhaly primárně pomocí headsetu Meta Quest 3. Testování zahrnovalo obě běžně dostupné metody připojení k počítači. První metodou bylo přímé spojení kabelem zaručující maximální propustnost obrazu a minimální latenci. Druhou metodou byl bezdrátový přenos přes lokální síť (***Air Link***), který uživateli poskytuje naprostou volnost pohybu za cenu mírné komprese obrazových dat.

6.2 Snímková frekvence a stabilita (FPS)

Nejdůležitějším ukazatelem použitelnosti aplikace ve virtuální realitě je stabilita snímkové frekvence. V enginu Unity bylo provedeno měření pro různé hustoty mřížky. Sledovaly se dva odlišné stavy. Prvním byl klidový stav, kdy uživatel graf

pouze pozoruje a prochází jím. Druhým stavem byla aktivní manipulace, kdy uživatel plynule táhne za posuvník parametru a aplikace musí v každém snímku generovat novou prostorovou síť.

Rozlišení sítě	Počet vrcholů	FPS v klidu	FPS při změnách parametrů
100×100	10 000	72	55
200×200	40 000	71	35
250×250	62 500	70	30
300×300	90 000	71	16

Tabulka 6.1: Závislost snímkové frekvence na hustotě sítě a uživatelské interakci.

Z naměřených hodnot (viz tabulka 6.1) vyplývá zásadní přínos architektury řízené událostmi. V klidovém stavu naráží aplikace na hardwarový limit obnovovací frekvence samotného headsetu (72 Hz). Tím se potvrzuje, že samotné vykreslení a zobrazení sítě s 90 000 vrcholy nedělá grafické kartě sebemenší problém. Výkon není limitován vykreslovacím vláknem.

Při aktivní manipulaci s parametry dochází k očekávanému snížení snímkové frekvence. Zde se projevuje výpočetní náročnost generování sítě na procesoru, zejména při iterování metodou bisekce. Zajímavým a klíčovým zjištěním však je, že uživatelský zážitek ve virtuální realitě zůstává plynulý i při naměřených hodnotách kolem 30 FPS.

Nutno ale podotknout, že měření probíhalo přímo ve vývojovém prostředí Unity Editor, které samo o sobě spotřebovává značnou část výkonu. Ve finálním přeloženém sestavení aplikace se výpočetní limity posouvají o něco výše.

6.3 Praktické ukázky vygenerovaných modelů

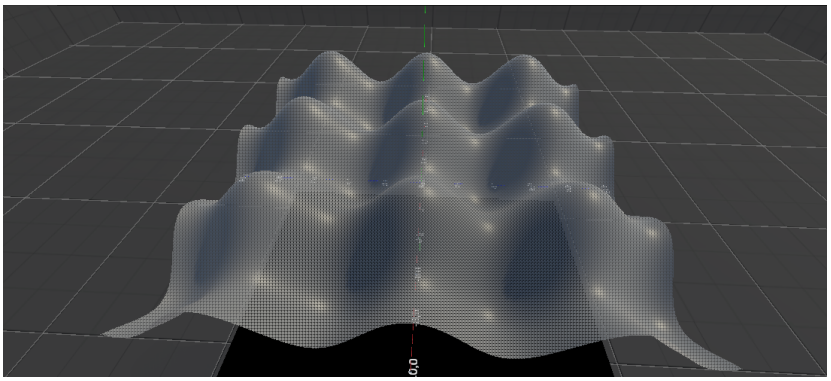
Pro ověření funkčnosti celého řešení byla aplikace testována na sadě různorodých matematických předpisů. Cílem bylo potvrdit schopnost analyzátoru správně interpretovat složitý uživatelský textový řetězec a následně demonstrovat vizuální stabilitu topologicky odlišných ploch.

6.3.1 Spojité plochy a dynamické parametry

Prvním testovacím scénářem byla komplexní goniometrická funkce definovaná předpisem $f(x, y) = a \cdot \sin(b \cdot x) + c \cdot \cos(d \cdot y)$. Tento předpis primárně ověřuje schopnost aplikace vykreslovat a upravovat spojité funkce.

Ve virtuální realitě vytváří tato rovnice plynulou zvlněnou plochu. Vykreslování a prostorová deformace sítě je zde bezproblémové i při velmi rychlé manipulaci s posuvníky (viz obrázek 6.1). Funkce totiž na celém svém definičním oboru neob-

sahuje žádné body nespojitosti. Procesor tak není zatěžován náročnými iteracemi vyhledávací metody bisekce.



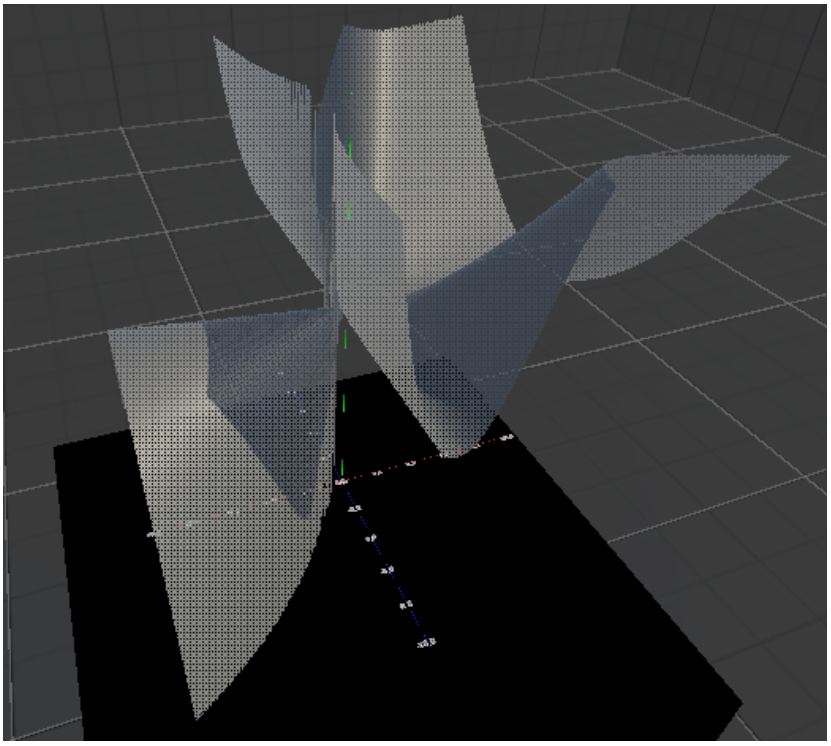
Obrázek 6.1: Vizualizace spojité goniometrické funkce se čtyřmi modifikovatelnými parametry (a, b, c, d).

6.3.2 Zpracování singularit a dělení nulou

Druhým příkladem zastupujícím extrémně problematické rovnice je lomená funkce s předpisem $f(x, y) = \frac{a \cdot x^2 \cdot b \cdot y^2}{c \cdot x^3 + d \cdot y^2}$. Tento matematický model představuje zatěžkávací zkoušku pro samotný generátor geometrie.

Graf v tomto případě obsahuje výškové skoky a složité singularity v místech, kde se hodnota jmenovatele blíží k nule. Na výsledném vygenerovaném modelu (viz obrázek 6.2) je jasně patrné fungování naprogramované topologické ochrany. Většinu nechtěných polygonů natahujících se přes propast generátor úspěšně detekuje a ořezává pomocí algoritmu bisekce. Při detailním zkoumání lze však v některých místech zpozorovat mírně zubaté okraje nebo ojedinělý polygon protínající oblast nespojitosti.

Tato chyba je primárně způsobena konečným rozlišením generované sítě a maximálním povoleným počtem iterací vyhledávacího algoritmu. Pokud je skok funkce příliš strmý a reálná vzdálenost bodů napříč propastí je menší než základní krok mřížky, algoritmus nedokáže tuto hranu s absolutní přesností izolovat. Snaha o úplné odstranění tohoto jevu by vyžadovala drastické zahuštění sítě nebo neúměrné navýšení počtu iterací. To by následně vedlo ke kritickému propadu snímkové frekvence. Z kontextu celkového tvaru plochy je nicméně pro pozorovatele naprosto zřejmé, že tyto chybné spojnice do grafu matematicky nepatří a nepůsobí tak při průzkumu natolik rušivě.



Obrázek 6.2: Ukázka lomené funkce.

Závěr

Cílem této práce bylo vyvinout aplikaci, která umožní vizualizovat a interaktivně zkoumat matematické funkce o dvou neznámých ve virtuální realitě. Aplikace by měla být schopna zpracovávat širokou škálu funkcí, včetně funkcí s body nespojitosti, a zároveň poskytovat plynulý uživatelský zážitek bez ohledu na komplexitu zobrazené geometrie.

7.1 Shrnutí dosažených výsledků

Aplikace úspěšně plní všechny stanovené cíle. Zcela bezproblémově zpracovává spojité funkce a s vysokou mírou spolehlivosti vizualizuje i funkce se singularitami. Tyto cíle byly dosaženy pomocí následujících přístupů a optimalizací:

- **Algoritmus *Shunting-yard* pro analýzu rovnic:** Tento algoritmus umožnil efektivní převod uživatelského textového vstupu do formy, která je snadno zpracovatelná pro vytváření abstraktního syntaktického stromu a jeho následné kompilace.
- **Kompilace pro rychlé vyhodnocování:** Vytvoření syntaktického stromu a jeho kompilace do mezikódu zrychluje výpočet jednotlivých vrcholů prostorové sítě na procesoru.
- **Algoritmus bisekce:** Implementace algoritmu bisekce zajišťuje detekci a ořezávání polygonů protínajících propasti, což umožňuje bezpečné zobrazování funkcí s body nespojitosti.
- **Optimalizace paměti a recyklace objektů:** Minimalizace alokací a využití existujících datových struktur urychluje překreslování grafu a výrazně snižuje zátěž procesu ***Garbage Collector***.
- **Architektura řízená událostmi:** Oddělení výpočtů od renderovací smyčky zajišťuje, že procesorový čas je využíván pouze v okamžicích změny dat. Díky tomu si aplikace udržuje maximální snímkovou frekvenci v klidovém stavu.

7.2 Možnosti budoucího rozšíření

Ačkoliv aplikace úspěšně plní svůj hlavní účel, nabízí se několik možností pro její budoucí rozšíření a optimalizaci:

- **Komplexnější zpracování singularit:** Ačkoliv aplikace aktuálně dokáže detekovat a odstranit většinu chybných polygonů, v místech nejsložitějších nespojitostí se stále objevují drobné vizuální artefakty.
- **Nezávislé sestavení:** V současné verzi je aplikace určena pro běh primárně přes počítač. Překlad do čistě nezávislého sestavení by naplno využil potenciál headsetů s vlastním výpočetním výkonem (jako je například Meta Quest).
- **Rozhraní pro zadávání funkcí:** Místo externí úpravy textového souboru by bylo uživatelsky přívětivější implementovat virtuální klávesnici přímo do aplikace, což by umožnilo zadávat a ukládat nové rovnice bez nutnosti sundávat headset.

Uživatelská příručka

Tato příloha slouží jako návod k ovládání aplikace v prostředí virtuální reality. Uživatelské rozhraní je navrženo tak, aby bylo co nejvíce intuitivní. Uživatel k pohybu a manipulaci s objekty využívá standardní pohybové ovladače podporované vývojovým rámcem XR Interaction Toolkit.

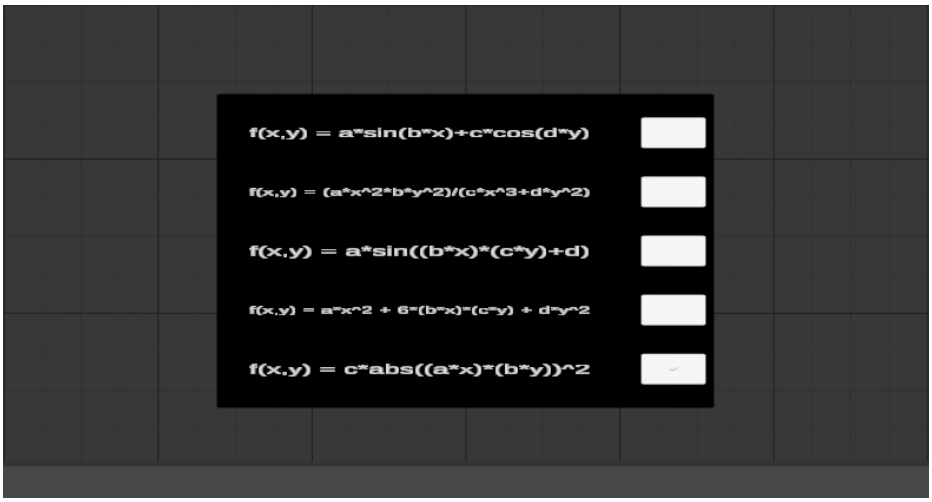


Obrázek A.1: Základní rozložení prvků a interakčních paprsků na virtuálním ovladači.

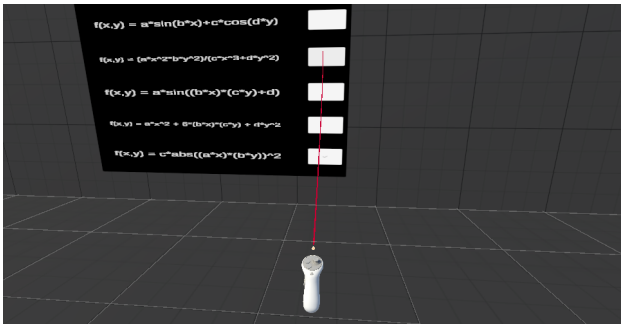
A.1 Hlavní tabule a výběr funkce

Po spuštění aplikace se uživatel ocitne ve virtuálním prostoru, kde na stěně se nachází hlavní interaktivní tabule. Na této ploše se nachází seznam všech dostupných matematických funkcí. Tyto funkce se dynamicky načítají z externího textového souboru.

Pro výběr konkrétní rovnice slouží virtuální laserový paprsek (***Raycast***). Tento paprsek (viz obrázek A.3) vychází nepřetržitě z virtuálních pohybových ovladačů. Uživatel jednoduše zamíří na požadovanou rovnici a stiskne `Trigger`. Okamžitě poté proběhne na pozadí výpočet a v prostoru se vygeneruje trojrozměrný model zvoleného grafu.



Obrázek A.2: Hlavní interaktivní tabule s nabídkou načtených matematických funkcí.



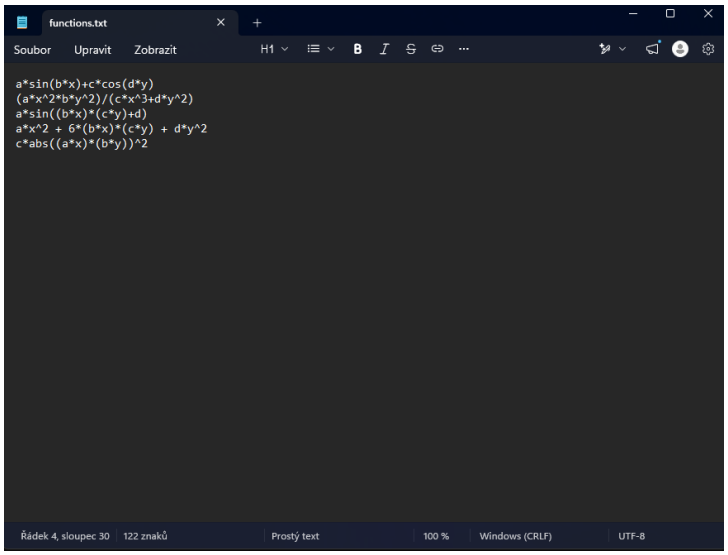
Obrázek A.3: Ukázka virtuálního laserového paprsku sloužícího k výběru funkce.

A.2 Úprava seznamu funkcí

Nabídka rovnic na hlavní tabuli není v aplikaci pevně zakódována. Veškeré matematické předpisy se dynamicky načítají z externího textového souboru. Tento přístup umožňuje uživatelům velmi snadno přidávat, upravovat nebo mazat vlastní matematické modely bez nutnosti jakkoliv zasahovat do zdrojového kódu.

Proces přidání nové funkce do aplikace je navržen s ohledem na maximální plynulost práce a skládá se z následujících kroků:

- **Lokalizace a otevření souboru:** Uživatel vyhledá příslušný textový soubor v kořenové složce aplikace a otevře jej v libovolném textovém editoru (například v programu Poznámkový blok).
- **Zápis rovnice:** Každá matematická funkce musí být zapsána výhradně na samostatném řádku. Aplikace plně podporuje standardní matematický zápis



Obrázek A.4: Ukázka struktury textového souboru s definicemi matematických funkcí.

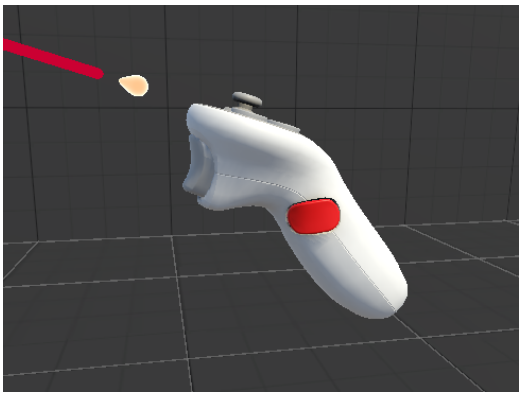
včetně závorek a běžných operací (například `sin`, `cos`, `sqrt`). Vstupní prostorové proměnné jsou striktně označovány písmeny x a y.

- **Využití vlastních parametrů:** Pokud uživatel do zápisu vloží libovolné neznámé znaky (například písmena a, b, c), analyzátor je při načítání automaticky rozpozná jako dynamické parametry. Pro tyto nalezené proměnné se posléze ve virtuální realitě samy vygenerují posuvníky v příručním menu.
- **Uložení a automatická aktualizace:** Jakmile uživatel textový soubor uloží, aplikace tuto změnu na pozadí okamžitě detekuje. Seznam na virtuální tabuli se automaticky překreslí.

A.3 Fyzická manipulace s modelem

Vygenerovaný matematický objekt představuje plně interaktivní prvek. Aplikace naplno využívá fyzikální povahu virtuální reality a umožňuje s grafem prostorově manipulovat následovně:

- **Přesun a rotace:** Uživatel může ke grafu přistoupit a stisknutím úchopového tlačítka (viz obrázek A.5) jej vzít přímo do ruky. Graf lze posléze libovolně otáčet, naklánět a přesouvat v prostoru pro detailní prozkoumání ze všech úhlů.
- **Změna měřítka:** Pro zkoumání asymptot nebo naopak celkového tvaru funkce je k dispozici režim obouručního škálování. Uživatel uchopí graf oběma



Obrázek A.5: Detail úchopového tlačítka na virtuálním ovladači.

rukama současně. Následným plynulým pohybem rukou od sebe se model zvětšuje. Pohybem rukou k sobě se model naopak zmenšuje.

- **Obnova pozice:** Může nastat situace, kdy uživatel nedopatřením upustí graf mimo vymezenou herní plochu, případně jej příliš zmenší. Pro tyto případy je k dispozici resetovací tlačítko (viz obrázek A.6), které pozici a měřítko modelu okamžitě obnoví.

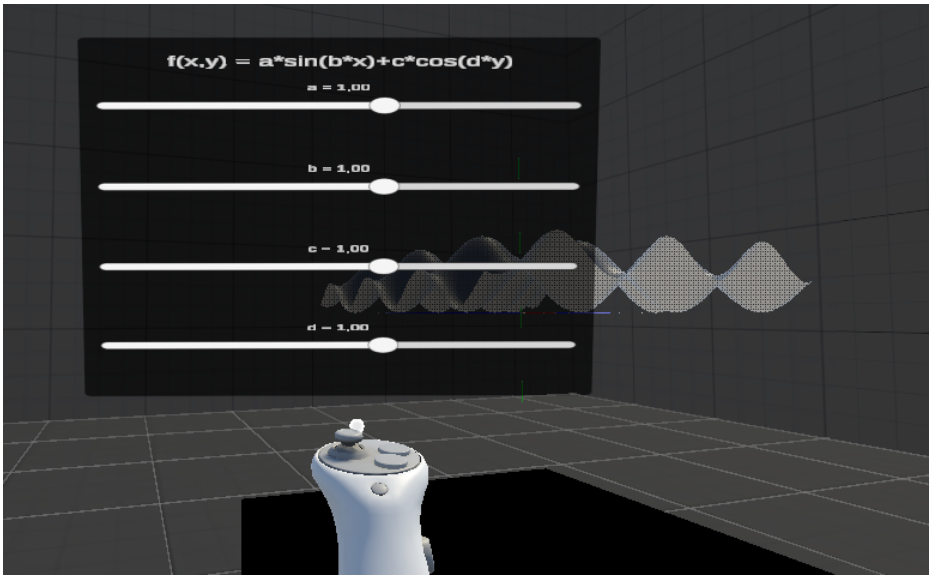


Obrázek A.6: Tlačítko pro okamžitý reset pozice a měřítka grafu.

A.4 Příruční menu a dynamické úpravy

Pro hlubší analýzu tvaru a chování funkce má uživatel k dispozici dvě interaktivní příruční menu. Toto rozhraní je ukotveno k virtuálnímu levému ovladači a vyvolává

se stisknutím určitého tlačítka. Každé z těchto menu slouží k jiné úpravě grafu.



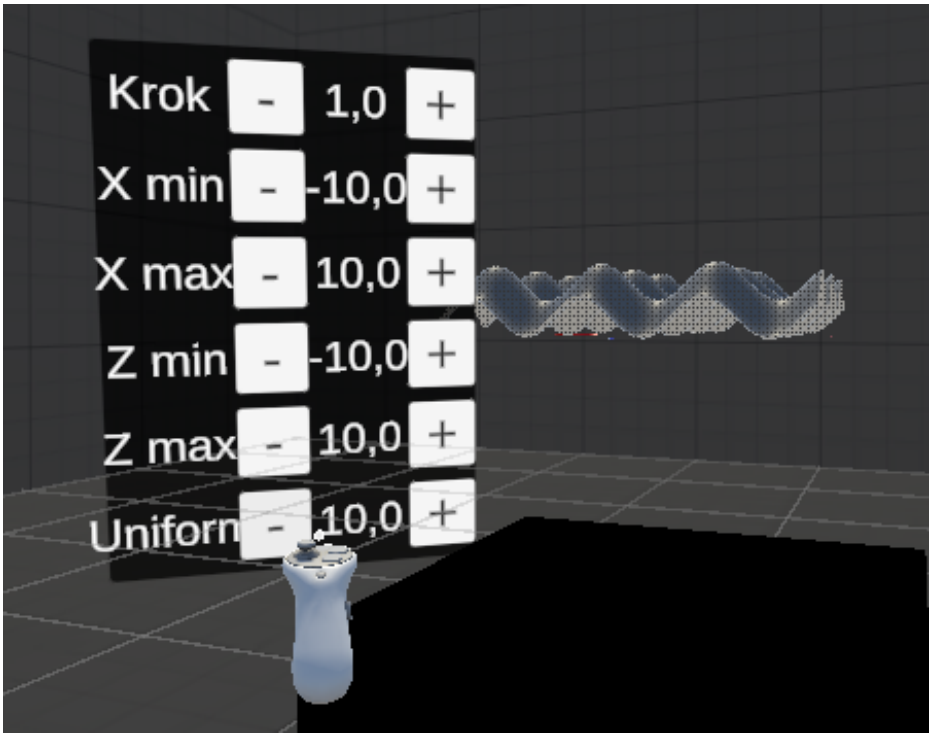
Obrázek A.7: Příruční menu ovladače pro změnu parametrů

První menu slouží k úpravě parametrů rovnice a uživatel jej vyvolá tlačítkem ukázaným na obrázku A.9. Nacházejí se zde dynamicky vygenerované posuvníky pro úpravu nalezených koeficientů (například parametrů a, b, c). Uživatel je může laserovým paprskem uchopit a plynule s nimi pohybovat. Každá změna hodnoty se okamžitě projeví na grafu.

Druhé menu upravuje nastavení definičního oboru a aktivuje se příslušným tlačítkem (viz obrázek A.10). Obsahuje krokovací tlačítka pro přesné určení hranic vykreslování na osách x a y. ***Uniform*** pak umožňuje měnit velikost definičního oboru symetricky do všech směrů současně.

A.5 Průzkum vnitřní topologie

Zásadní výhodou virtuální reality je volnost pohybu. Uživatel není vázán pouze na statický pohled zvenčí. Díky implementaci oboustranného grafického materiálu může do vygenerovaného modelu fyzicky vstoupit. Získá tak zcela unikátní perspektivu pro pochopení složitých prostorových útvarů a vnitřních struktur zkoumaných matematických funkcí.



Obrázek A.8: Příruční menu ovladače pro změnu definičního oboru.



Obrázek A.9: Tlačítko pro vyvolání menu s parametry funkce.



Obrázek A.10: Tlačítko pro vyvolání menu s nastavením definičního oboru.

Bibliografie

[Dij61] DIJKSTRA, Edsger. *Algol 60 Translation: An Algol 60 Translator for the X1 and Making a Translator for Algol 60* [online]. 1961. [cit. 2026-05-03]. Tech. zpr. Mathematisch Centrum. Dostupné z: http://www.cs.utexas.edu/users/EWD/MCReps/MR35.PDF.

[Mic24] MICROSOFT. *System.Linq.Expressions Namespace* [online]. 2024. [cit. 2026-05-03]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.linq.expressions?view=net-10.0.

[Uni25] UNITY TECHNOLOGIES. *XR Interaction Toolkit Manual* [online]. 2025. [cit. 2026-05-03]. Dostupné z: https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.3/manual/index.html.